

Computer-Use Automation System: public reference review

Purpose and reading of the evidence

This brief evaluates three **public, evidence-rich reference implementations** against interface.ai’s *Computer-Use Automation System* take-home. The assignment’s critical thread is not merely browser control: it is **a real LLM discovery run → a typed, reviewable capability → deterministic, no-model replay → explicit outcomes/safety/evidence → a human takeover of the same live session**. It also asks for a credible path across legacy surfaces and many tenant-specific application instances.

The ordering below follows the supplied review rubric and the inspected, commit-pinned sources. It is **not an official leaderboard and does not claim these are “top submissions”** ; no official leaderboard was available. Two repositories document themselves as interface.ai take-home implementations. The first is best treated as a **close, independent reference implementation of the assignment pattern**, rather than a confirmed candidate submission: its public documentation describes the system, but the reviewed material does not establish an official submission relationship. 1 2 14 21

How to use this brief: borrow the architectural principles and testable invariants—not code, prompts, fixtures, artifacts, screenshots, or wording. Each repository is a local/synthetic demonstration, not a production banking integration.

At-a-glance comparison — demonstrated features only

Legend: **Yes** = demonstrated at the pinned revision for the stated scope; **Partial** = a meaningful seam/control exists but material scope or safety limitations remain; **Not evidenced** = no working proof in the reviewed source. This is deliberately narrower than design intent and does not infer production readiness.

Assignment-relevant capability	bgivenb/computer-use-automation	Niramay-Kelkar/hands-off	Niram replay
Real LLM-driven observe → decide → act run against a live UI	Yes — committed OpenAI exploration evidence 8	Yes — committed Claude/Playwright trajectory 19	Yes — Claud 27
Typed, versioned, reviewable capability with inputs, outputs, locators, and checks	Yes 4	Yes 18	Yes 2!

Deterministic replay with no model decision loop	Yes 5	Yes 17	Yes 24
Structured success, business-outcome, and failure/escalation handling	Yes 4 5	Yes 17 19	Yes 24
Configurable allowlists/risk control plus redacted durable text evidence	Partial — screenshots are not pixel-redacted 2 10	Partial — masking is configuration/shape driven 15 19	Partial — retain extrac
Structured run evidence and a rich failure/handoff signal	Yes 7 13	Yes 19	Yes 24
Human takes over the same live browser session and replay resumes without repeating the effectful action	Yes 12	Yes — replay scope 17 19	Yes —
Implemented non-web/desktop surface	Not evidenced 2	Not evidenced 15	Not e
Executed multi-tenant reuse beyond one target	Not evidenced 2	Yes — controlled second tenant; target binding is reprinted 19 20	Not e
Production-grade remote operator authorization, durable distributed control, and full PII media governance	Not evidenced 2	Not evidenced 15	Not e

The table highlights an important conclusion: all three satisfy the **core single-web-surface lifecycle** credibly, but none proves a deployable banking platform. The strongest differentiators are (1) policy/evidence/session-control rigor in the first reference and (2) a bounded but real second-tenant exercise in the second. 2 15 22

1. bgivenb/computer-use-automation — strongest all-round independent reference

GitHub: [bgivenb/computer-use-automation](https://github.com/bgivenb/computer-use-automation) at commit 1499c252c662fd237c82025dab9460404b484e92 . 1

What it is and how it works

This TypeScript/Playwright system works against a synthetic, deliberately awkward legacy-style bank-servicing fixture. Its demonstrated capability accepts a synthetic member identifier and account nickname, extracts a member name and savings balance, and reaches a final-review checkpoint before an irreversible account-creation action; the fixture includes iframe/nested-table conditions and named error states. 1 3

Its vertical slice is architecturally clean. A provider-neutral model driver powers a bounded discovery loop that observes the page/frame state, selected controls, state digest, and screenshot; requests structured OpenAI Responses output; runs policy checks; acts on the live surface; records events; and validates checkpoints. A successful trace is compiled with a trusted, hand-authored profile into a strict JSON capability. Replay then consumes the saved artifact and invocation inputs, rather than a model driver, and executes stored commands/locators in closed-world order. 2 4 5 6

Concrete, source-proven strengths

- **Real discovery evidence is distinguished from fixtures.** The committed exploration summary identifies openai-responses , gpt-5.4-mini , explore mode, and nine model calls; the evidence manifest and README separate that provider-backed material from credential-free/model-free demonstrations. This is stronger than a prose-only claim, though it remains creator-supplied evidence. 1 7 8
- **The capability is a serious invoker contract, not a macro.** Its strict Zod schema includes a schema version, identity/revision/provenance, application family/version/surface, sensitivity-labelled typed inputs/outputs, business outcomes, permissions, ordered steps/branches, robustness-rationalized locator bundles, and success conditions. The compiler parameterizes discovery samples and rejects leaked values and generated/id selectors. 4 6
- **Replay is meaningfully deterministic and fails closed on locator ambiguity.** The executor has no model dependency and reports modelCalls: 0 ; it evaluates stored locator strategies in order, accepts exactly one match, verifies postconditions/final conditions, type-checks outputs, and returns success, business outcome, or structured failure. Committed scenarios cover not-found, a declared transient recovery, permission failure, handoff, and changed-input replay. 5 6 9
- **The safety posture is unusually strong for a take-home.** Runtime and artifact origin/route/action allowlists intersect; action risk is raised from the observed target semantics; irreversible steps are denied or routed to intervention; and the target is rechecked immediately before use. Browser controls constrain non-allowlisted network/WebSocket traffic, service workers, and downloads, while the event writer performs structured redaction before persisted JSONL. 10 11 2

- **Handoff is modeled as exclusive ownership, not a vague pause.** RunSession holds a single browser context/page and moves control through automation → none → human → none → automation ; the coordinator serializes human actions on that live page and rejects resume until a declared checkpoint passes. The evidence verifier also validates hashes, schemas, event ordering, run IDs, and model-call counts across the committed package. 12 13 9

Gaps and risks relative to the assignment

- **Scope is web-only and synthetic.** SurfaceAdapter and tenant-profile/drift-quarantine concepts are credible seams, but the artifact currently permits only web ; there is no desktop adapter, tenant overlay resolver, drift service, distributed lease, or authenticated remote operator routing. The assignment allows these as design seams, but they are not demonstrated features. 2 4
- **Discovery is intentionally guided, not open-ended capability induction.** The compiler combines the model's successful mechanical trajectory with trusted profile-defined inputs, outcomes, permissions, checkpoints, and branch catalog. That is a defensible safety boundary, but it should be described honestly: it does not prove that a model can safely infer an entire bank capability contract from only natural language. 2 4 6
- **The financial-data boundary is not production-safe.** Structured events are deeply redacted, but screenshots are not pixel-redacted before storage or provider transmission. The repository itself limits the design to synthetic data until it has screenshot DLP, retention/deletion controls, encryption, access audit, and data-governance review. 2 11
- **Evidence integrity is internal consistency, not independent provenance.** The verifier protects a declared evidence package against unnoticed changes and semantic mismatch, but cannot cryptographically prove that a particular provider call occurred; the manifest records collection at earlier source revisions. 7 13
- **The operator and scale plane is explicitly demo-grade.** A loopback, in-process coordinator is not a replacement for authenticated operator identity, notification, multi-operator arbitration, durable browser workers, or distributed fencing. 2 12

Superior-design lessons to retain

1. Make the **capability artifact the versioned executable control plane**, while treating the model trajectory as evidence and input to review—not as the production program.

4 6

2. Use a **two-key authority model**: the effective permission set is the intersection of runtime/tenant policy and artifact policy, and bind approval to the resolved target immediately before action. 10 11
 3. Preserve explicit **success** / **business_outcome** / **failure** contracts and bounded, declared recovery rather than permitting model improvisation during replay. 5 6
 4. Treat handoff as a **lease plus verified resume condition** on the original session. 12
 5. Preserve evidence verification, but label it accurately as **integrity checking**, then add independent attestation and secure evidence management for production. 13
-

2. Niramay-Kelkar/hands-off — strongest bounded multi-tenant proof

GitHub: [Niramay-Kelkar/hands-off](#) at commit `c360e9964efd23e979288da78d68f871096beecf` .14

What it is and how it works

This is an explicitly documented interface.ai take-home implementation for a local legacy-style member-search application. Discovery launches a live Chromium page, supplies a redacted accessibility-tree observation to a Claude tool-use loop, executes one constrained UI tool action per turn through Playwright, and stores the trajectory and screenshots. The committed trajectory connects its goal, tool calls, outputs, and terminal completion to the compiled artifact's source run. 14 16 19

The compiler deliberately separates **mechanically observed facts** (actions, parameters, locators, extracts, checkpoints) from separately authored policy (capability metadata, risk class, outcomes, guardrails, evidence policy, escalation behavior). Replay takes the compiled capability and inputs only, applies locator/action/route policy, and returns typed success, business outcome, or hard failure. Its ACT → SETTLE → CHECK split means a timeout or post-human resume re-observes/re-checks rather than blindly clicking again. 15 17 18

Concrete, source-proven strengths

- **The policy/mechanics split is unusually sound.** A happy-path trajectory can prove that a locator and action worked once; it cannot establish what is safe, what business outcomes are legitimate, or which recovery is permitted. The compiler's separate policy layer keeps that distinction reviewable. 15 18
- **The replay state machine prevents a common financial-automation failure.** ACT is distinct from SETTLE and CHECK, so retry or handoff resume does not repeat an action that may already have taken effect. The evidence includes success,

MEMBER_NOT_FOUND , ACCESS_DENIED , dialog escalation/resume, and eight successful fresh-browser replays. 17 19

- **The capability is typed and inspectable.** The Pydantic model and compiled JSON expose schema/version and source-run metadata, target/surface, typed inputs/outputs, locators, checkpoints, outcome policy, guardrails, evidence, and escalation configuration. 18
- **It provides the strongest concrete tenant-reuse evidence in this set.** Tenant B changes branding, CSS, DOM shell, classes, result structure, and database; committed material exercises success, not-found, and an escalation path there. The mechanics/locators/checkpoints remain byte-identical, while target binding is repointed for Tenant B. This is a well-scoped compatibility demonstration, not a claim of automatic generalization. 19 20
- **The minimal handoff is real rather than mocked.** A SQLite session record tracks status/owner/step/context; a second Playwright client attaches to the existing browser through CDP, dismisses a dialog, invokes the real resume flow, and replay completes from the same page. 17 19

Gaps and risks relative to the assignment

- **Do not describe this as an unchanged, universal artifact across tenants.** Tenant B requires a copy with target.entry_point and target.app repointed because the allowlisted origin is part of the artifact. Moreover, both tenants deliberately retain the accessibility role/name contract. This is strong bounded evidence, but not a tenant registry, overlay system, drift detector, or third-tenant demonstration. 15 19 20
- **The only complete end-to-end target flow is read-only lookup.** The mutation/confirmation gate is proven using a synthetic capability fixture, not a real mutating target-app flow; discovery/compiler are not covered by the CI smoke path. 15 19
- **Ambiguous locators can act incorrectly.** Replay takes .first after resolving a strategy instead of requiring unique cardinality, so a duplicate accessible name after subtle UI drift could select the wrong control rather than fail safely. 17
- **The handoff control plane is local and insecure by production standards.** The Flask/SQLite/CDP path lacks authentication, authorization, durable worker ownership, active notification, resilient browser lifetime, and a granular audit of every human browser action. Discovery exhaustion terminates rather than escalating into this handoff path. 15 19
- **Data handling and orchestration remain demo-scale.** Redaction is useful but configuration/DOM-shape driven rather than a comprehensive sensitive-data classifier;

no durable audit/metrics/retention plane, queue, concurrency model, or desktop implementation is shown. 15 19

Superior-design lessons to retain

1. Keep **observed mechanics** separate from **authored/reviewed safety and business policy**; bind both to provenance, reviewer approval, and an effective tenant/version scope. 15 18
2. Adopt **ACT → SETTLE → CHECK**, then strengthen it with action idempotency class, external-effect correlation IDs, an irreversible barrier, and postcondition reconciliation. 17
3. Preserve ranked locator telemetry, but require **unique resolution plus contextual assertions** before acting; use the telemetry as a drift signal, not a substitute for fail-closed safety. 17
4. Treat tenant reuse as **base capability + explicit tenant binding/overlay + compatibility evidence**, never as a branding-only claim. 19 20

3. Niramay-Kelkar/discovery-replay — strongest compact lifecycle reference with candid scale limits

GitHub: [Niramay-Kelkar/discovery-replay](#) at commit 2581347e722b4f7b2c9cbf24d52a35118db8107f . 21

What it is and how it works

This is also explicitly documented as an interface.ai take-home implementation. It drives a deliberately hostile, server-rendered mock bank UI with nested tables and generic CSS but a usable accessibility tree. A Claude tool-use discovery agent observes an accessibility snapshot, makes one constrained tool action at a time against the live Playwright page, and saves a trajectory. A compiler creates a capability with a mechanical layer plus hand-authored policy; the `Replayer` then runs that frozen capability through **ACT → SETTLE → CHECK** and emits `Success` , `BusinessOutcome` , `HardFailure` , or `PendingEscalation` . 21 22 23 24 26

On unrecognized state, replay writes an escalation record to SQLite, keeps the browser open, allows headed/manual or CDP access to the same tab, and resumes with verification only after an operator action. The committed evidence records the discovery run, replay results, a five-run stability summary, and a handoff that proceeds from an unrecognized dialog to operator action and successful completion. 24 27 28

Concrete, source-proven strengths

- **A genuine model-driven discovery trace is inspectable.** The committed trajectory identifies the model, records a seven-turn action/extract/done path and screenshots, while the discovery code shows the constrained live-browser tool loop. 23 27
- **It has a strong schema/provenance story.** The Pydantic models and capability define schema/capability versions, typed input/output definitions, actions, ranked locators, checkpoints, outcomes, guardrails, escalation policy, discovery provenance, and named policy authorship. 25 26
- **It encodes one-shot actions structurally.** ACT is done once; timeouts and resumed human handoff use SETTLE/CHECK. This is a superior default to generic retries where a prior click might have created an external effect. 24
- **The result taxonomy is practical.** Not-found, access-denied, and supervisor-review conditions are modeled separately from hard failure and unknown-dialog escalation, rather than making every non-happy path a crash. 24 26
- **Its same-session proof is concrete.** The handoff log records the escalation, CDP intervention by an operator, an explicit statement that ACT was not re-invoked, and subsequent success. 24 28

Gaps and risks relative to the assignment

- **Its heterogeneity and tenant architecture are largely design-only.** The report explicitly says there is no `Surface` protocol, tenant registry/configuration, tenant overlays, credential isolation, drift enforcement, queue/worker deployment model, or desktop adapter. The code is Playwright-bound and `iframe/within` support is incomplete. 22 24
- **Locator reliability is not uniformly fail-closed.** The detail result uses a prefix locator with `nth: 0`; changed ordering or several matching rows could select an unintended first result. Other locator types are open in the schema but not evidenced by live runs. 22 24 26
- **Mutating workflow safety is not exercised end to end.** The confirmation path is tested with an altered/synthetic read-only artifact, so risk classification and real external-effect handling remain unproven. Discovery evidence intentionally keeps synthetic sample values, which would not meet production PII retention needs. 22 24
- **Its human console is minimal by design.** Resume is unauthenticated and local/CDP-based; there is no remote secure co-browse, notification, ownership lease, or multi-operator locking. 22 24 29

- **The replay evidence should not be over-read as scale evidence.** Five sequential success runs demonstrate repeatability on the fixture, not reliability across versions, tenants, concurrent load, or actual banking operations. 22 30

Superior-design lessons to retain

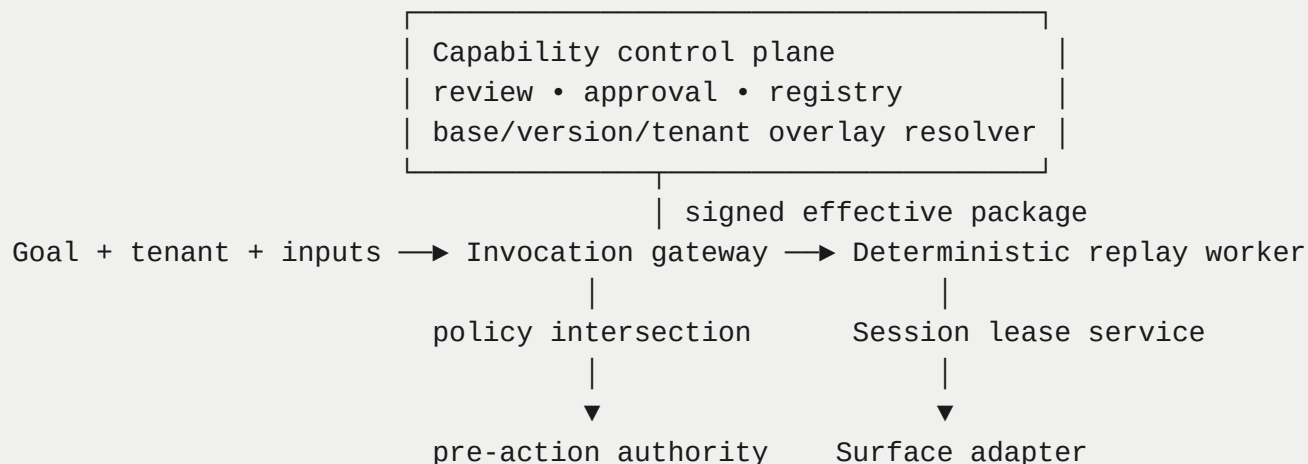
1. Let the compiler enforce that the model can supply **trajectory evidence**, but cannot silently manufacture policy, risk classification, or failure semantics. 22 26
2. Make one-shot ACT and re-enterable verification a **state-machine invariant**, not an engineering convention. 24
3. Keep business outcomes, escalation, and hard failure as **distinct first-class caller contracts**. 17
4. For human takeover, retain the same browser/tab and resume only with reconciliation—but add authenticated leases, fencing, and complete action auditing. 24 28 29

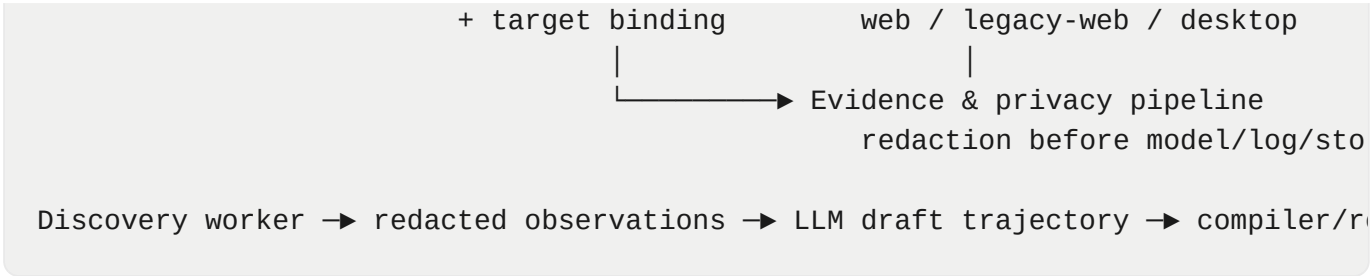
How to build something superior

A superior candidate response should be **narrower than a platform claim but deeper than a browser-agent demo**: implement one real, read-oriented legacy-web workflow end to end; demonstrate deliberate failure and handoff cases; and make the seams production-credible. The objective is not to out-feature the references. It is to combine their best invariants while closing the gaps that matter most in a financial environment.

Original architecture: signed capability packages on a surface-neutral execution core

Plain Text





1. A capability package has four deliberately separate layers

1. **Mechanical flow** — typed commands, ordered state transitions, locator portfolios, frame/window scope, pre/postconditions, extraction bindings, and explicit action idempotency class.
2. **Execution policy** — risk class, permissions, route/origin allowlists, outcome taxonomy, recovery budget/backoff, escalation conditions, confirmation/approval rules, and evidence policy. This layer is authored/reviewed, never inferred from a green run.
3. **Compatibility scope** — vendor product/family, semantic version range, approved surface types, fingerprint/probe expectations, and references to immutable evidence.
4. **Tenant overlay** — entry point, allowed origins, credential reference, localization/alias differences, narrowed policy, and locator additions or overrides. It may only narrow authority or add a reviewed fallback; it cannot silently broaden global policy.

Every package should be schema-versioned, signed after review, and immutable by digest. The invoker resolves **base vendor capability** → **compatible version revision** → **tenant overlay**, emits the resulting effective-package hash, and refuses an unknown combination. This incorporates the strongest mechanical/policy boundary from both Niramay references and the artifact-first contract from `computer-use-automation`, while making the currently conceptual tenant model executable. 2 15 22

2. Introduce a real SurfaceAdapter boundary

The artifact uses portable vocabulary— `observe`, `resolve`, `act`, `settle`, `check`, `capture`, and `human_control` —rather than Playwright selectors as its fundamental API. Concrete adapters provide:

Adapter	Primary targeting	Fallbacks	Required runtime proof before action
Modern/legacy web	accessibility role/name, label, text, DOM context	scoped structural CSS; OCR/visual anchor for hostile markup	unique cardinality, ancestor/context signature, route/frame identity

Frames/embedded web	adapter-scoped frame path plus the web portfolio	visual anchor in declared frame	frame fingerprint and target re-resolution
Desktop	OS accessibility role/name/value/window identity	visual/OCR anchor plus coordinate only inside a signed region	window/process identity, unique control match, focus proof

A locator portfolio records **why** each strategy is safe, expected cardinality, required surrounding context, and a health score. Unlike the `.first` weakness in the second reference or `nth: 0` weakness in the third, any non-unique risky locator becomes `PendingEscalation` or a diagnosable drift failure—never an arbitrary click. ¹⁷ ²⁴

3. Make replay an effect-aware state machine

Use the following deterministic phases for every step:

1. **PREFLIGHT:** validate signed package, typed inputs, effective tenant policy, credential reference, version/fingerprint compatibility, and session lease.
2. **RESOLVE + AUTHORIZE:** resolve a unique target, calculate the effective policy intersection, inspect the actual action destination/form semantics, and bind a short-lived target fingerprint.
3. **ACT (at most once):** record a correlation ID and any known external-effect key. Read-only steps may be retried only according to their declared safe class.
4. **SETTLE + RECONCILE:** wait for a bounded declared condition; after a timeout or handoff, query observable state and correlate the prior effect instead of reissuing it.
5. **CHECK + CLASSIFY:** produce `Success`, declared `BusinessOutcome`, recoverable state, `PendingEscalation`, `DriftQuarantine`, or structured `HardFailure` with step/expected/observed/evidence references.

For writes, require an explicit irreversible barrier: an authenticated, action-specific human approval or institution-approved workflow token immediately before ACT. A capability may reach a confirmation screen in the take-home; proving that the barrier blocks the final write is more valuable than pretending full autonomous financial mutation is safe.

4. Treat data minimization as a precondition—not a logging afterthought

Before an observation is sent to an LLM or persisted, a privacy gateway should apply structured field classification, allowlisted accessibility text, DOM/value suppression, and region-aware screenshot redaction. Store encrypted evidence references, not raw content,

with tenant-scoped keys, retention/deletion schedules, access audit, and break-glass approval. For a take-home, document that all data are synthetic and demonstrate that **the exact screenshot/log presented to the model is already redacted**—closing the key screenshot gap in the first reference and the synthetic-raw trajectory caveat in the third. 22

5. Make human handoff a durable security protocol

A browser-session service owns the live browser independently of the replay process. It issues a lease with a fencing token and makes controller state explicit: automation → released → operator → released → automation . The operator receives an authenticated, expiring intervention link with RBAC and step-specific authority. Every human browser action, page transition, before/after snapshot digest, and resume attestation is appended to an immutable audit stream. Resume is permitted only if the session lease/fencing token is current and reconciliation proves the declared resume checkpoint.

This keeps the important same-session and no-repeated-ACT properties proven by all three references while fixing their local SQLite/CDP/unauthenticated control-plane limitations. 12 17 24 29

6. Use drift management as a product feature

At invocation, run low-cost compatibility probes: vendor/version signals, key accessibility/visual fingerprints, expected route/frame topology, and locator cardinality. Aggregate locator-resolution outcomes by tenant/version. When a threshold crosses, quarantine only the affected overlay, route new invocations to intervention or a known-good revision, and schedule bounded rediscovery plus human review. Use canary replays with synthetic records before promoting a new base artifact or tenant overlay.

Priority choices for a focused take-home

Priority	Build and demonstrate	Why it wins on the assignment
P0 — must ship	One hostile local web proxy; one genuine LLM discovery trajectory; signed/versioned capability; model-free replay with unique locator resolution; success + not-found + permission/dialog outcome; one rich failure signal	Directly proves the required vertical slice rather than a design-only platform
P0 — safety invariant	Effective policy intersection; pre-action target fingerprint;	Demonstrates judgment in the parts that fail dangerously in

	redaction <i>before</i> LLM/log use; ACT → SETTLE → CHECK; typed terminal result union	banking workflows
P0 — human proof	Existing-session takeover with lease states, authenticated mock operator identity, human action audit, and verification- only resume	Exceeds a superficial “pause and restart” handoff without building a full co-browse product
P1 — portability proof	One <code>SurfaceAdapter</code> interface plus web adapter; a tenant overlay resolver; a deliberately changed second tenant with compatibility probes	Turns scale/heterogeneity discussion into bounded evidence without attempting desktop automation
P1 — assurance	Evidence manifest with artifact/source hashes, verifier, test matrix, and a clear statement of what integrity evidence cannot prove	Makes review repeatable and avoids conflating self-produced logs with independent attestation
P2 — document, do not fake	Desktop adapter contract, remote operator architecture, queue/worker design, DLP/retention/KMS plan, drift quarantine rollout	Meets the brief’s design-for- scale requirement while being candid about what was not implemented

Final recommendation

Use `bgivenb/computer-use-automation` for its **artifact discipline, intersecting policy ceilings, target binding, evidence verification, and lease model**; use `hands-off` for its **policy/mechanics separation, ACT → SETTLE → CHECK behavior, and bounded tenant-B proof**; and use `discovery-replay` for its **compact typed result taxonomy and same-session, verification-only handoff evidence**. 2 15 22

The superior original solution is not a collage of these repositories. It is a coherent system that visibly enforces four claims: **the model only discovers; policy is reviewed; replay never re-reasons or repeats an uncertain effect; and a tenant-scoped, privacy-safe, leased session is always accountable to a human operator.**

GitHub source references

All sources below are pinned to the revisions reviewed. Source citations support repository-specific statements in this brief.

- [1] README — scope, target, evidence guide
- [2] Architecture report and explicit limits
- [3] Synthetic target and capability specification
- [4] Typed capability and result contracts
- [5] Model-free replay implementation
- [6] Discovery/compiler boundary
- [7] Evidence manifest and provenance
- [8] Provider-backed exploration summary
- [9] Changed-input zero-model replay summary
- [10] Runtime and artifact policy enforcement
- [11] Browser restrictions, target checks, and session runtime
- [12] Same-session intervention coordinator
- [13] Evidence-package verifier
- [14] README — take-home implementation
- [15] Design report and scoped limitations
- [16] Live Claude/Playwright discovery loop
- [17] Replay phases, policy, and locator execution
- [18] Compiled capability artifact
- [19] Curated discovery, replay, and handoff evidence
- [20] Second-tenant fixture
- [21] README — take-home implementation
- [22] Architecture report and omissions
- [23] Claude discovery loop
- [24] Deterministic replay, policy, and handoff
- [25] Capability models
- [26] Versioned capability and policy
- [27] Committed discovery trajectory
- [28] Committed same-session handoff evidence
- [29] Operator console
- [30] Five-run replay stability summary